Министерство науки и высшего образования Российской Федерации

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

«Национальный исследовательский университет ИТМО»

(Университет ИТМО)

Факультет программной инженерии и компьютерной техники

Образовательная программа Проектирование и разработка систем больших данных

Направление подготовки 09.04.04 Программная инженерия

Дисциплина Аналитика больших данных

О Т Ч Е Т

по домашнему заданию 1

«Построение хранилища и витрин данных»

Обучающиеся:

Ермаков Тимофей Сергеевич, группа P4135

Перов Виктор Валерьевич, группа P4135

Преподаватель: доцент, к.т.н.

Королёва Юлия Александровна

Санкт-Петербург

2026

Цель работы

Освоить подходы к построению хранилища данных: проектирование витрины данных на основе нормализованной модели, загрузку данных из внешних источников в отдельную схему хранилища и организацию ETL-процессов на Apache Airflow.

Задание

1. По модели данных магазина изделий ручной работы (master_product, master, client, order) написать запрос, формирующий витрину данных по заказчикам с разбивкой по месяцам.
2. Создать отдельную схему external_source, подключить её к хранилищу и разложить данные из трёх CSV-файлов по таблицам.
3. Организовать журналирование изменений доменных таблиц в logs и синхронизацию данных хранилища с MongoDB.

Исходный код

https://github.com/mgfallen/adb-itmo

Архитектура

Весь стенд поднимается одной командой docker compose up. В него входят PostgreSQL - хранилище, MongoDB - приёмник синхронизации задания 3, и Apache Airflow в режиме standalone - среда запуска ETL-процессов.

Схема и начальные данные создаются SQL-миграциями из каталога migrations. Миграции монтируются в /docker-entrypoint-initdb.d, поэтому при первом старте контейнера БД создаётся автоматически; для ручного наката и отката есть скрипт scripts/migrate.sh. Доменные таблицы задания 1 лежат в схеме public, внешние данные задания 2 - в схеме external_source, журнал изменений задания 3 - в таблице logs.

Витрина задания 1 реализована Go-модулем handmade_market: SQL-запрос хранится в repo.go, результат собирается в структуру ReportRow драйвером pgx v5. Для запроса написаны интеграционный и unit-тесты, которые запускаются в GitHub Actions при каждом push.

В dags/ лежат два DAG: load_external_source - загрузка трёх CSV-файлов в external_source, и sync_mongo - синхронизация доменных таблиц и журнала изменений в коллекции MongoDB.

Задание 1. Витрина данных по заказчикам

Модель данных

Модель магазина изделий ручной работы состоит из пяти таблиц:

- client - заказчики: идентификатор, ФИО, адрес, дата рождения, электронная почта;
- master - мастера ручной работы;
- product - товары, для витрины категорией товара выступает поле type;
- master_product - таблица «мастера и товары»: у каждого мастера своя цена на товар, ключ - пара (master_id, product_id);
- order - заказы: заказчик, мастер, товар, даты создания и доставки, статус.

Статусы заказа: created (создан), in_progress (в процессе изготовления), completed (завершён); факт оплаты помечается статусом paid. Поскольку цена товара зависит от мастера, стоимость заказа берётся из master_product по паре (master_id, product_id).

Требования к витрине

Витрина содержит по одной строке на каждого заказчика за каждый отчётный период (месяц) со следующими показателями:

- идентификатор записи, идентификатор заказчика, ФИО, адрес, дата рождения и электронная почта заказчика;

- сумма всех оплаченных заказов клиента;
- количество заказов за месяц и средняя стоимость одного заказа за месяц;
- медианное время в днях от момента создания заказа до его завершения за месяц;
- самая популярная категория товаров и идентификатор самого популярного мастера за месяц;
- количество созданных заказов, заказов в процессе изготовления, завершённых и незавершённых за месяц;
- отчётный период - год и месяц.

Реализация запроса

Запрос написан для PostgreSQL и разбит на общие табличные выражения, каждое из которых отвечает за свою часть расчёта.

order_details. Заказы дополняются категорией товара (product.type) и ценой из master_product. Соединение с master_product - LEFT JOIN: если у пары (мастер, товар) цена не задана, заказ всё равно попадает в витрину, просто не участвует в денежных агрегатах. Здесь же каждому заказу назначается отчётный период - to_char(order_date, 'YYYY-MM').

client_monthly. Агрегаты по заказчику и месяцу. Сумма всех оплаченных заказов считается коррелированным подзапросом по статусу paid без привязки к месяцу - по условию это сумма всех оплаченных заказов клиента, а не только за период. Количество заказов в каждом статусе считается через SUM(CASE WHEN ...). Медианное время выполнения - PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY delivery_date - order_date); заказы без даты доставки (ещё не завершённые) в расчёт не попадают, так как NULL игнорируется агрегатом.

popular_category и popular_master. Заказы месяца группируются по категории товара и по мастеру, ROW_NUMBER() нумерует группы по убыванию количества заказов. При равном количестве берётся первая по

алфавиту категория или мастер с меньшим идентификатором - выбор детерминированный.

В итоговом SELECT результаты соединяются по заказчику и периоду, популярные категория и мастер подтягиваются строками с номером 1. Идентификатор записи - ROW_NUMBER() OVER (ORDER BY reporting_period, client_id), порядок строк не зависит от плана выполнения. Полный текст запроса приведён в листинге 1.

Листинг 1 – Запрос для построения витрины данных по заказчикам

```
WITH order_details AS (
  SELECT
    o.order_id,
    o.client_id,
    o.master_id,
    o.product_id,
    p.type AS product_type,
    mp.price,
    o.order_date,
    o.delivery_date,
    o.status,
    TO_CHAR(o.order_date, 'YYYY-MM') AS reporting_period
  FROM "order" o
  JOIN client c ON o.client_id = c.client_id
  JOIN product p ON o.product_id = p.product_id
  LEFT JOIN master_product mp ON o.master_id = mp.master_id AND o.product_id = mp.product_id
),
client_monthly AS (
  SELECT
    od.client_id,
    od.reporting_period,
    (
      SELECT SUM(mp_all.price)
      FROM "order" o_all
      JOIN master_product mp_all
        ON o_all.master_id = mp_all.master_id AND o_all.product_id = mp_all.product_id
      WHERE o_all.client_id = od.client_id AND o_all.status = 'paid'
    ) AS total_paid_amount,
    COUNT(od.order_id) AS monthly_orders_count,
    AVG(od.price) AS avg_monthly_order_price,
    SUM(CASE WHEN od.status = 'created' THEN 1 ELSE 0 END) AS count_created,
    SUM(CASE WHEN od.status = 'in_progress' THEN 1 ELSE 0 END) AS count_in_progress,
    SUM(CASE WHEN od.status = 'completed' THEN 1 ELSE 0 END) AS count_completed,
    SUM(CASE WHEN od.status != 'completed' THEN 1 ELSE 0 END) AS count_uncompleted,
    PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY (od.delivery_date - od.order_date)) AS
median_completion_days
  FROM order_details od
  GROUP BY od.client_id, od.reporting_period
),
popular_category AS (
  SELECT client_id, reporting_period, product_type,
      ROW_NUMBER() OVER (PARTITION BY client_id, reporting_period ORDER BY COUNT(*) DESC,
product_type) as rn
  FROM order_details
  GROUP BY client_id, reporting_period, product_type
),
popular_master AS (
  SELECT client_id, reporting_period, master_id,
      ROW_NUMBER() OVER (PARTITION BY client_id, reporting_period ORDER BY COUNT(*) DESC, master_id) as
rn
```

```
  FROM order_details
  GROUP BY client_id, reporting_period, master_id
)
SELECT
  ROW_NUMBER() OVER (ORDER BY cm.reporting_period, c.client_id) AS record_id,
  c.client_id,
  c.name AS client_name,
  c.address AS client_address,
  c.birthday AS client_birthday,
  c.email AS client_email,
  cm.total_paid_amount,
  cm.monthly_orders_count,
  cm.avg_monthly_order_price,
  cm.median_completion_days,
  pc.product_type AS popular_product_category,
  pm.master_id AS popular_master_id,
  cm.count_created,
  cm.count_in_progress,
  cm.count_completed,
  cm.count_uncompleted,
  cm.reporting_period
FROM client c
JOIN client_monthly cm ON c.client_id = cm.client_id
LEFT JOIN popular_category pc
  ON c.client_id = pc.client_id AND cm.reporting_period = pc.reporting_period AND pc.rn = 1
LEFT JOIN popular_master pm
  ON c.client_id = pm.client_id AND cm.reporting_period = pm.reporting_period AND pm.rn = 1;
```

Проверка запроса

1) интеграционный тест repo_test.go: в тестовой БД создаётся схема модели, два заказчика и шесть заказов в разных статусах за сентябрь и октябрь 2026 года, после чего результат запроса поэлементно сверяется с ожидаемым (go test ./...). 2)
2) unit-тест repo_mock_test.go: ответ запроса мокается библиотекой pgxmock, проверяется сборка структур ReportRow, в том числе обработка NULL-полей «популярная категория» и «популярный мастер».
3) ручная проверка на демо-данных - результат в таблице 1.

Таблица 1 – Результат построения витрины на демо-данных

Показатель	**Запись 1**	**Запись 2**
Идентификатор заказчика	2	1
ФИО заказчика	Пётр Петров	Иван Иванов
Адрес заказчика	Казань	Москва
Дата рождения заказчика	1990-01-01	1995-03-15
Электронная почта	petr@example.com	ivan@example.com
Сумма оплаченных заказов	1 000,00	1 800,00
Количество заказов	3	3
Средняя стоимость заказа	1 033,33	1 000,00
Медиана выполнения, дн	0,5	2
Популярная категория товаров	украшения	украшения

ID популярного мастера	2	1
Создано заказов	1	0
В изготовлении	1	0
Завершено	0	1
Не завершено	3	2
Отчётный период	2026-09	2026-10

Результаты совпадают с ручным расчётом. У заказчика Иванова в октябре три заказа ценой 1000, 800 и 1200, оплачены первые два - сумма оплаченных 1800, средняя стоимость заказа (1000 + 800 + 1200) / 3 = 1000. Время выполнения заказов - 2, 4 и 1 день, медиана 2. Чаще всего заказывали украшения (2 заказа), все два раза у мастера 1. У заказчика Петрова в сентябре из трёх заказов оплачен только один, поэтому завершённых заказов 0, а незавершённых 3. Даты доставки указаны у двух заказов (1 и 0 дней), медиана 0,5 - PERCENTILE_CONT для чётного числа значений берёт середину между двумя центральными.

Задание 2. Загрузка данных из внешних CSV-файлов

Внешний источник представлен тремя CSV-файлами, каждый загружается в свою таблицу схемы external_source (таблица 2). В целевых таблицах, помимо полей из условия задания, сохранены и исходные колонки файлов - так любую строку можно проследить до исходной записи источника.

Таблица 2 – загрузка CSV-файлов по таблицам external_source

Исходный файл	**Целевая таблица**	**Поля из задания**	**Строк загружено**
user_order_log.csv	external_source.user_order_log	Date_time, Customer_id, Quantity, Payment_amount	8
user_activity_log.csv	external_source.user_activity_log	Date_time, Customer_id	6
customer_research.csv	external_source.customer_research	Date_id, Sales_qty, Sales_amt	8

Загрузку выполняет один DAG load_external_source с тремя независимыми задачами (PythonOperator) - по задаче на файл, они могут работать параллельно. Каждая задача вызывает функцию copy_csv_to_pg (листинг 2): таблица очищается TRUNCATE и заполняется командой COPY ... FROM STDIN с заголовком CSV. TRUNCATE и COPY выполняются в одной транзакции, поэтому при ошибке копирования таблица откатывается к прежнему содержимому. DAG идемпотентен: его можно перезапускать сколько угодно раз, дублей в таблицах не будет.

Листинг 2 – Задача загрузки CSV (фрагмент dags/load_external_source.py)

```
def copy_csv_to_pg(filename: str, table: str, columns: list[str]) -> None:
    """Очищает целевую таблицу и копирует CSV по указанным колонкам."""
    conn = psycopg2.connect(PG_DSN)
    try:
        with conn.cursor() as cur:
            cur.execute(f"TRUNCATE TABLE {table}")
            with open(os.path.join(CSV_DIR, filename), "rb") as csv_file:
                cur.copy_expert(
                    "COPY {table} ({cols}) FROM STDIN WITH (FORMAT csv, HEADER true)".format(
                        table=table, cols=", ".join(columns)
                    ),
                    csv_file,
                )
        conn.commit()
    finally:
        conn.close()
```

Баг, который мы нашли и пофиксили - в первой версии DAG в списках колонок COPY для обеих таблиц логов была пропущена первая колонка id, которая есть в CSV-файлах. COPY читает колонки позиционно, поэтому загрузка падала с ошибкой BadCopyFileFormat: extra data after last expected column. T.e. customer_research, где список колонок совпадал с файлом, загрузилась, а обе таблицы логов остались пустыми после TRUNCATE. После добавления колонки id в DAG отработал успешно, см. таблицу 2.

Задание 3. Журнал изменений и синхронизация с MongoDB

Журнал изменений logs

Таблица logs хранит все изменения, внесённые в доменные таблицы: идентификатор записи журнала, имя таблицы, тип операции (INSERT/UPDATE/DELETE, ограничение CHECK), снимок изменённой

строки в формате JSONB и время изменения. Заполнение автоматическое: на каждую доменную таблицу навешен триггер AFTER INSERT OR UPDATE OR DELETE, вызывающий общую функцию log_table_change() (листинг 3). Снимок строки берётся как to_jsonb(COALESCE(NEW, OLD)): после вставки и обновления это новая версия строки, после удаления - старая.

Листинг 3 – Журнал изменений и триггер

```
CREATE TABLE IF NOT EXISTS logs (
   log_id     BIGSERIAL PRIMARY KEY,
   table_name TEXT        NOT NULL,
   operation  TEXT        NOT NULL,
   payload    JSONB       NOT NULL,
   changed_at TIMESTAMPTZ NOT NULL DEFAULT now(),
   CHECK (operation IN ('INSERT', 'UPDATE', 'DELETE'))
);

-- Универсальная функция аудита: пишет снимок NEW после INSERT/UPDATE
-- и снимок OLD после DELETE.
CREATE OR REPLACE FUNCTION log_table_change() RETURNS trigger AS $$
BEGIN
   INSERT INTO logs (table_name, operation, payload)
   VALUES (TG_TABLE_NAME, TG_OP, to_jsonb(COALESCE(NEW, OLD)));
   RETURN COALESCE(NEW, OLD);
END;
$$ LANGUAGE plpgsql;

-- Триггер навешивается на каждую доменную таблицу, пример:
CREATE TRIGGER trg_order_audit
   AFTER INSERT OR UPDATE OR DELETE ON "order"
   FOR EACH ROW EXECUTE FUNCTION log_table_change();
```

Синхронизация с MongoDB

DAG sync_mongo состоит из четырёх независимых задач: clients, masters, orders и logs - по задаче на коллекцию, они выполняются параллельно. Задача читает таблицу из PostgreSQL через psycopg2 и переносит строки в одноимённую коллекцию MongoDB через pymongo. Документ записывается командой replace_one с upsert=True по натуральному ключу (client_id, master_id, order_id, log_id): повторный запуск обновляет существующие документы, а не создаёт дубли. В каждый документ пишется update_time - время последней синхронизации. Журнал logs переносится целиком, payload сериализуется в строку JSON.

Проверка: после запуска DAG в базе handmade_market MongoDB созданы коллекции clients (2 документа), masters (2), orders (6) и logs (копия журнала изменений). Пример документа из коллекции orders:

{"_id": 1, "client_id": 1, "master_id": 1, "product_id": 1, "order_date": "2026-10-01", "delivery_date": "2026-10-03", "status": "paid", "update_time": "2026-10-24T00:14:14"}

Тестирование и CI

Логика витрины покрыта двумя тестами. Интеграционный TestGetMonthlyReportIntegration поднимает собственную схему с тестовыми данными в реальном PostgreSQL и сверяет каждую строку витрины с ожидаемой; строка подключения задаётся переменной TEST_DATABASE_DSN, при недоступности БД тест пропускается. Юнит-тест TestGetMonthlyReportMock проверяет сборку результата без БД: ответ запроса подменяется библиотекой pgxmock.

Тесты запускаются в GitHub Actions (.github/workflows/ci.yml) при каждом push и pull request. Job go-tests поднимает PostgreSQL, выполняет go vet и go test. Job dags-syntax компилирует DAG-файлы python -m py_compile.

Выводы

В ходе работы создан единый Docker-стенд с PostgreSQL, MongoDB и Airflow. По модели магазина изделий ручной работы реализована витрина по заказчикам с разбивкой по месяцам (CTE-запрос + Go/pgx), три CSV-источника загружаются в схему external_source командой COPY, изменения доменных таблиц журналируются триггерами в logs и синхронизируются в MongoDB. Витрина покрыта интеграционным и unit-тестами, CI проверяет тесты и синтаксис DAG на каждый push. На реальном прогоне DAG загрузки был найден и исправлен баг расхождения списка колонок COPY с заголовком CSV-файла.